

DFI 5.2 Port & Drive-Logic Reference

Post-phaser DFI adapter, one channel, **DDR5** (DDR1–4 later). Source: DFI 5.2 (Cadence, Oct 2024). Config: $N_RANKS=2$, $DQ=32b$ sub-channel, $BL=16$, $GEAR \in \{1, 2, 4\} = \text{phases}/\text{DFI-clk}$, $CA=14b$. Dir = from MC (out) or from PHY (in).

1 Clocking — one DFI clock, three reference domains

Every DFI signal is clocked by the single **dfi_clk** (DFI 5.2 Table 2). There is no separate “DDR clock” the MC logic sits in — the PHY generates the DRAM CK behind the phase interface. Each signal is only *associated* with a reference domain for its timing/phase relationship:

- **Command clock**: **dfi_address**, **dfi_cs**, **dfi_cke**, **dfi_odt**, **dfi_reset_n**, **dfi_parity_in**, **dfi_alert_n**, **dfi_2n_mode**, **dfi_cs_geardown**, **dfi_dram_clk_disable** (+ legacy **dfi_act_n/ras_n/cas_n/we_n/bank/bg/cid**).
- **Data clock**: **dfi_wrdata***, **dfi_rddata***, **dfi_wck_***.
- **Control clock**: **dfi_init_***, **dfi_*_freq_ratio**, **dfi_freq_fsp**, **dfi_frequency**, **dfi_ctrlupd_***, **dfi_phyupd_***, **dfi_error***, **dfi_lp_***.

Frequency ratio (GEAR): in a 1:N system each signal is replicated per phase with a **_pN** suffix (**_p0** may drop the suffix); $N=GEAR$ command/data phases ride one **dfi_clk** edge. **Baseline (STAGE 4)**: **matched**, **dfi_clk == mc_clk** — the whole post-phaser adapter is ONE clock domain (**mc_clk**); the reference domains only tell you which phase a bus beat lands on. So your instinct is right that the enables are “DDR-clock” by reference, but physically the MC drives them on **dfi_clk**, per phase.

The single clock is the SLOW clock (**dfi_clk = CK/GEAR**); the $N \times$ is **SPATIAL**, not a faster clock. At 1:4 **dfi_clk** runs at $CK/4$ and *each edge drives 4 CA phases in parallel* (**dfi_address_p0..p3**, **dfi_cs_p0..p3**); the PHY serializes those 4 onto the DRAM CA at full CK. So the MC *never* runs at CK — it runs at $CK/GEAR$ and emits a whole bundle’s worth of CA/enables per cycle:

- 1:1 $\rightarrow$ 1 phase/edge (**dfi_clk=CK**)
- 1:2 $\rightarrow$ 2 phases/edge (**dfi_clk=CK/2**)
- 1:4 $\rightarrow$ 4 phases/edge (**dfi_clk=CK/4**)

This is exactly the phaser’s N-winner bundle (STAGE 4). A DDR5 command is 2-UI, so it occupies 2 of the N phase slots; at 1:4 that means up to 2 two-cycle commands per edge (**[C C A A]**) or a mix with 1-cycle PRE (fig_55). **DFI_CMD_ENCODE** builds the N phase-slots; the PHY does the CK-rate serialization.

2 Command interface (Command-clk ref, all _pN per phase)

Signal	Dir	Width	Ref clk	Meaning
dfi_address(_pN)	out	14	cmd	CA bus. DDR5 command is CA-encoded here (2-UI cmds span 2 beats); MC preserves bit order.
dfi_cs(_pN)	out	$N_RANKS=2$	cmd	Chip select (active low), one per rank/CS. Asserted on the command’s phase(s).
dfi_cke(_pN)	out	$N_RANKS=2$	cmd	Clock enable per rank (power-down / self-refresh entry-exit).
dfi_odt(_pN)	out	$N_RANKS=2$	cmd	On-die-termination control per rank.
dfi_reset_n(_pN)	out	1	cmd	DRAM reset (init).
dfi_2n_mode(_pN)	out	1	cmd	2N command timing (hold $CA \geq 1$ cmd-clk after CS for DDR5 w/o RCD).
dfi_cs_geardown(_pN)	out	1	cmd	DDR5 CS geardown mode select.
dfi_dram_clk_disable(_pN)	out	per-clk	cmd	Gate the DRAM clock (clock-stop / low power).
dfi_parity_in(_pN)	out	1	cmd	CA parity forwarded to PHY (tparin_lat after cmd).
dfi_alert_n(_pN)	in	1	cmd	PHY $\rightarrow$ MC alert (CA-parity / write-CRC error), tphy_paritylat.
dfi_act_n/ras_n/cas_n/we_n(_pN)	out	1 each	cmd	<i>Legacy DDR3/4</i> discrete command encode. Unused for DDR5 (CA on dfi_address).
dfi_bank/bg/cid(_pN)	out	2/3/?	cmd	<i>Legacy</i> DDR3/4 bank/BG; cid = chip-ID for 3DS.

3 Write data interface (Data-clk ref, _pN per phase)

Signal	Dir	Width	Ref clk	Meaning
dfi_wrdata(_pN)	out	$2 \cdot DQ=64$	data	Write data, driven tphy_wrdata cycles AFTER dfi_wrdata_en , for as long as en held.
dfi_wrdata_en(_pN)	out	per-slice	data	Marks valid wrdata/mask/crc window. Generated by the WL launcher (tphy_wrlat from CAS-W).
dfi_wrdata_mask(_pN)	out	$2 \cdot DQ/8=8$	data	Byte mask (DM) or write-DBI. Bit <i>k</i> masks byte <i>k</i> of dfi_wrdata .
dfi_wrdata_cs(_pN)	out	PHY_RANK_W	data	Which rank the write data targets (host-timing compensation).
dfi_wrdata_crc(_pN)	out	DATA_W/8	data	Link CRC per byte-lane, same timing as dfi_wrdata .
dfi_wrdata_ecc(_pN)	out	(opt)	data	Write-data link ECC (if enabled).

4 Read data interface (Data-clk ref)

Signal	Dir	Width	Ref clk	Meaning
dfi_rddata(_wN)	in	2:DQ=64	data	Read data from PHY (_wN = per read-word phase).
dfi_rddata_en(_pN)	out	per-slice	data	MC tells PHY the read-capture window. Generated by the RL line (tphy_rdlat from RD).
dfi_rddata_valid(_wN)	in	per-slice	data	PHY → MC: this beat of dfi_rddata is valid (drives RD-capture into RD_SRAM).
dfi_rddata_cs(_pN)	out	PHY_RANK_W	data	Rank targeted for the read (host-timing).
dfi_rddata_dnv(_wN)	in	per-slice	data	Data-not-valid marker within a returned word.
dfi_rddata_dbi(_wN)	in	DATA_W/8	data	Read DBI (if enabled).
dfi_rddata_crc(_wN)	in	DATA_W/8	data	Read link CRC.

5 Status / Update / Error / Low-power (Control-clk ref, single-phase)

Signal	Dir	Width	Ref clk	Meaning
dfi_init_start	out	1	ctrl	MC starts init / requests a frequency change.
dfi_init_complete	in	1	ctrl	PHY: init done / freq-change ack. Gates the DFI-mux (init→scheduler).
dfi_cmd_freq_ratio	out	2	ctrl	Command frequency ratio (1:1/1:2/1:4) = GEAR select.
dfi_data_freq_ratio	out	2	ctrl	Data frequency ratio.
dfi_freq_fsp	out	1-2	ctrl	Frequency set-point (which MR timing set). Changes only at init / freq-change.
dfi_frequency	out	W	ctrl	Target operating frequency code.
dfi_ctrlupd_req	out	1	ctrl	MC requests a PHY update window.
dfi_ctrlupd_ack	in	1	ctrl	PHY grants the ctrl update.
dfi_phyupd_req	in	1	ctrl	PHY requests an update (e.g. periodic retrain).
dfi_phyupd_ack	out	1	ctrl	MC grants the PHY update.
dfi_phyupd_type	in	2	ctrl	Kind of PHY update.
dfi_error	in	1	ctrl	PHY error strobe.
dfi_error_info	in	W×4	ctrl	Error source (valid only while dfi_error).
dfi_lp_ctrl_req/ack/wakeup	out/in/out	1	ctrl	Low-power control-path handshake.
dfi_lp_data_req/ack/wakeup	out/in/out	1	ctrl	Low-power data-path handshake.

6 Post-phaser drive logic — encode after the phaser (decision)

Question: generate the DFI enables by (a) a DFI-encode stage after the phaser, or (b) let the scheduler drive every enable?

Decision: (a) encode after the phaser. The scheduler / phaser output is legality + placement (*what & which phase*); DFI framing (*the exact per-phase enables and CA beats*) is a separate encode. Reasons:

- The scheduler must stay selection-only; loading it with per-phase **dfi_cs**/CA-beat and **wrdata_en**/**rddata_en** timing would couple two unrelated concerns and bloat it.
- The enables are **deterministic delays off the CAS commit**, already owned by the data path (STAGE 5): **dfi_wrdata_en** from the WL launcher (tphy_wrlat), **dfi_rddata_en** from the RL line (tphy_rdlat). They are not scheduling decisions.
- DDR5 CA is 2-UI — the encoder is the natural place to spread one command over 2 beats and stamp **dfi_cs** on the right phase(s).

Blocks after the phaser (all in dfi_clk == mc_clk)

- DFI_CMD_ENCODE** (comb + 1 reg): phaser bundle → per-phase {**dfi_address_pN**, **dfi_cs_pN**}. Maps the DRAM command to the DDR5 CA-bus beats and asserts **dfi_cs** on the command's phase; fills idle phases with DES (CS high).
- WL_LAUNCH** (shift line, STAGE 5): at CAS-W commit, arm a tphy_wrlat delay; on expiry drive **dfi_wrdata_en_pN**, then stream **dfi_wrdata_pN**+**dfi_wrdata_mask_pN** (+CRC) for the burst from **WD_SRAM**.
- RD_CAP** (shift line): at RD commit, arm tphy_rdlat; assert **dfi_rddata_en_pN**; capture **dfi_rddata_wN** on **dfi_rddata_valid_wN** into **RD_SRAM**.
- DFI_MUX** (init override): selects init-FSM vs scheduler-encoder as CA driver; **sel** = **dfi_init_complete** (the only hard override, STAGE 9).
- ALERT/ERR monitor**: watch **dfi_alert_n** (write-CRC/parity retry window) and **dfi_error** (stub, STAGE 13/20).

Sideband (Control-clk): **dfi_init_***, **dfi_*_freq_ratio**, **dfi_ctrlupd_***, **dfi_phyupd_*** are driven by the maintenance / freq-change FSM, not the per-command encode.

7 DDR1–4 portability (to fill later)

Parametric (reload widths / values): GEAR, DFI Data/Address width, N_RANKS, CS/CKE/ODT widths, timing params (tphy_wrlat, tphy_rdlat, tphy_wrdata). **Structural:** DDR3/4 use the *discrete* command encode (**dfi_act_n/ras_n/cas_n/we_n** + **dfi_bank/bg**) instead of the DDR5 CA-bus-on-**dfi_address**, so **DFI_CMD_ENCODE** gets a per-gen encoder variant; DDR5-only signals (**dfi_cs_geardown**, **WCK**, **MRDIMM mux**, per-slice read valid) drop out. Clock model (single **dfi_clk**, 3 ref domains) is identical across all gens.